



AIR CARGO ANALYSIS

SQL Course-End Project

Data Acquisition and Manipulation Using SQL

Prepared by:

Anushua Ghosh

Tool: MySQL Workbench | April 2026

Certification: Data Acquisition and Manipulation Using SQL

Project Overview

Air Cargo is an aviation company providing passenger and freight transportation services. The goal of this project is to analyze and optimize operational data using SQL to enhance customer satisfaction and business efficiency.

This project focuses on three key areas:

1. Identifying regular customers for personalized offers
2. Analyzing the busiest routes to optimize aircraft allocation
3. Preparing detailed ticket sales and revenue reports

Dataset Description

Four CSV datasets were imported into the AirCargo database in MySQL Workbench:

Table	Key Columns	Records	Role
customer	customer_id (PK), first_name, last_name, date_of_birth, gender	50	Master customer info
passengers_on_flights	customer_id (FK), route_id (FK), aircraft_id, seat_num, class_id, travel_date	50	Flight travel records
routes	route_id (PK), flight_num, origin_airport, destination_airport, aircraft_id, distance_miles	49	Route master data
ticket_details	customer_id (FK), aircraft_id, class_id, no_of_tickets, price_per_ticket, brand, p_date	50	Ticket purchase details

Database Setup

The AirCargo database was created and all four datasets were imported as structured tables:

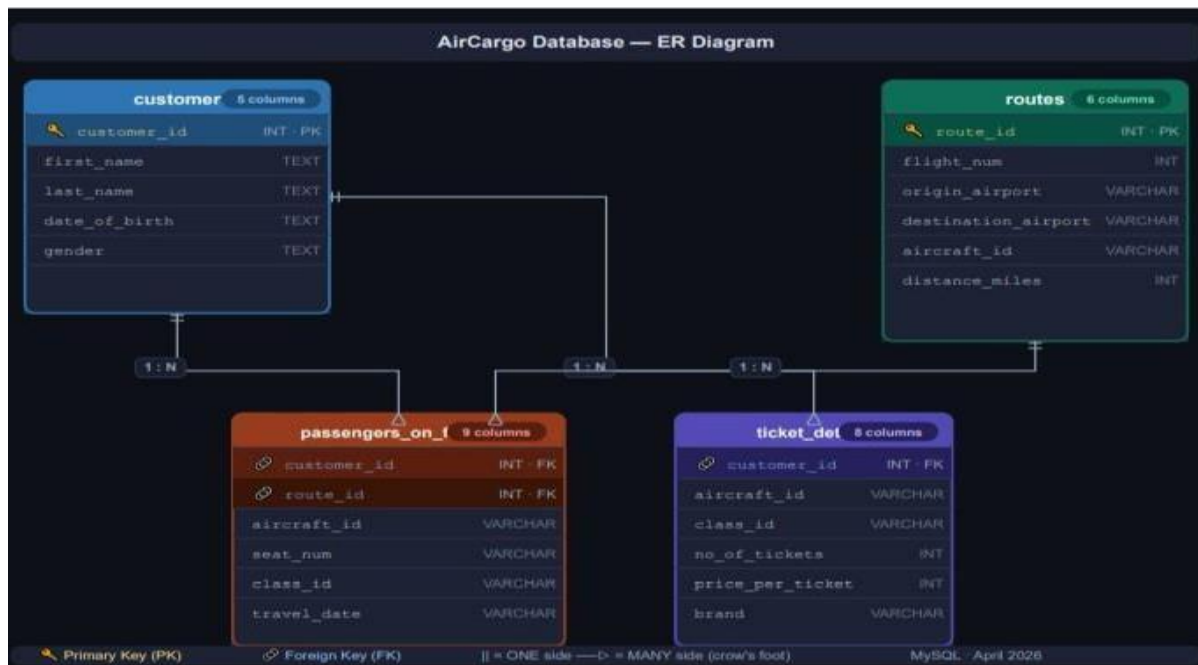
```
-- Step 1: Create the database
CREATE DATABASE AirCargo;
USE AirCargo;

-- Step 2: Import tables via MySQL Workbench Table Import Wizard
-- customer.csv --> customer
-- passengers_on_flights.csv --> passengers_on_flights
-- routes.csv --> routes
-- ticket_details.csv --> ticket_details
```

ER Diagram

The ER Diagram below illustrates the relationships between the four tables in the AirCargo database. Primary keys (PK) and foreign keys (FK) define the referential integrity between entities.

→ AirCargo — Entity Relationship Diagram
4 Tables · 3 Relationships · MySQL | Data Acquisition and Manipulation Using SQL



Relationship	Type	Description
customer → passengers_on_flights	1 : N	One customer can take many flights
customer → ticket_details	1 : N	One customer can buy many tickets
routes → passengers_on_flights	1 : N	One route can have many passengers

SQL Queries & Results

Query 1 — Passengers on Routes 1 to 25

Objective:

Retrieve all passengers who have traveled on route IDs between 1 and 25.

SQL Code:

```
SELECT *  
FROM passengers_on_flights  
WHERE route_id BETWEEN 1 AND 25;
```

Result:

✓ Approximately 25 rows returned, showing all travel details for routes 1 to 25.

Observation:

💡 The BETWEEN clause provides an inclusive range filter. This helps identify popular short-haul routes and their passenger distribution.

Query 2 — Business Class Passengers and Revenue

Objective:

Count the total number of tickets sold and calculate total revenue generated from Business class.

SQL Code:

```
SELECT SUM(no_of_tickets) AS num_passengers,  
       SUM(no_of_tickets * price_per_ticket) AS total_revenue  
FROM ticket_details  
WHERE class_id = 'Business';
```

Result:

✓ Passengers: 13 tickets sold | Total Revenue: Rs.6,034

Observation:

💡 Using SUM(no_of_tickets) gives the accurate count of passengers, not just booking records. Business class contributes Rs.6,034 out of Rs.15,369 total revenue (~39%).

Query 3 — Customer Full Name using CONCAT

Objective:

Display the full name of all customers by combining first name and last name.

SQL Code:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name  
FROM customer;
```

Result:

✓ 50 rows returned with full names of all registered customers.

Observation:

💡 CONCAT() is used to merge two columns into a single readable display column, useful for reports and dashboards.

Query 4 — Registered Customers Who Booked Tickets

Objective:

Identify customers who have both registered in the system and purchased at least one ticket.

SQL Code:

```
SELECT DISTINCT c.customer_id, c.first_name, c.last_name
FROM customer c
JOIN ticket_details t
ON c.customer_id = t.customer_id;
```

Result:

✓ 41 unique customers found who have both registered and purchased tickets. 9 customers registered but never booked.

Observation:

💡 INNER JOIN filters only matched records. DISTINCT removes duplicates from customers with multiple ticket bookings. 9 inactive customers are a direct business opportunity for win-back campaigns.

Query 5 — Emirates Customers — First Name, Last Name & ID

Objective:

Retrieve customer IDs, first names and last names for customers who have traveled with Emirates.

SQL Code:

```
SELECT DISTINCT c.customer_id,
               c.first_name,
               c.last_name
FROM customer c
JOIN ticket_details t
  ON c.customer_id = t.customer_id
WHERE t.brand = 'Emirates';
```

Result:

✓ 14 unique customers found who have traveled with Emirates airline. DISTINCT was added to remove duplicate entries as the same customer can purchase multiple Emirates tickets.

Observation:

💡 Without DISTINCT, 18 rows were returned with duplicate customer records. After adding DISTINCT, 14 unique Emirates customers were identified. Filtering by brand allows Air Cargo to perform airline-specific customer analysis.

Query 6 — Economy Plus Customers Using Subquery

Objective:

Identify customers who have traveled in Economy Plus class using a subquery.

SQL Code:

```
SELECT first_name, last_name
FROM customer
WHERE customer_id IN (
  SELECT customer_id
  FROM passengers_on_flights
  WHERE class_id = 'Economy Plus'
);
```

Result:

✓ All customers who flew in Economy Plus class are listed with their first and last names.

Observation:

💡 The subquery approach is efficient for existence checks. Inner query finds customer_ids from passengers_on_flights, outer query retrieves names from customer table. IN() checks if each customer_id is in that list.

Query 7 — Revenue Check Using IF Clause**Objective:**

Determine whether total ticket revenue has crossed Rs.10,000 using the IF() clause.

SQL Code:

```
SELECT SUM(no_of_tickets * price_per_ticket) AS total_revenue,
       IF(SUM(no_of_tickets * price_per_ticket) > 10000,
          'Yes - Target Achieved',
          'No - Target Not Met') AS revenue_status
FROM ticket_details;
```

Result:

✓ Total Revenue: Rs.15,369 | Status: Yes - Target Achieved! Revenue exceeds target by 53%.

Observation:

💡 Total revenue of Rs.15,369 exceeds the Rs.10,000 threshold. IF() is a conditional expression that allows inline business logic inside SQL queries — similar to Excel IF formula.

Query 8 — Create and Grant Access to a New User**Objective:**

Create a new MySQL user and grant full database privileges on the AirCargo database.

SQL Code:

```
-- Create a new user
CREATE USER 'aircargo_user'@'localhost' IDENTIFIED BY 'SecurePass@123';

-- Grant all privileges on AirCargo database
GRANT ALL PRIVILEGES ON AirCargo.*
TO 'aircargo_user'@'localhost';

-- Apply the changes
FLUSH PRIVILEGES;
```

Result:

✓ User 'aircargo_user' created successfully and granted all privileges on AirCargo database.

Observation:

💡 User management is a critical part of database administration. GRANT ALL PRIVILEGES gives complete CRUD access. In production, restrict to specific privileges (SELECT, INSERT, etc.) for security — following the Principle of Least Privilege.

Query 9 — Maximum Ticket Price per Class (Window Function)

Objective:

Find the maximum ticket price for each travel class using a window function.

SQL Code:

```
SELECT class_id,
       price_per_ticket,
       MAX(price_per_ticket) OVER (PARTITION BY class_id) AS max_price
FROM ticket_details;
```

Result:

✓ Each row shows its class's maximum price alongside the individual ticket price, without collapsing rows.

Observation:

💡 Window functions (OVER + PARTITION BY) perform aggregations while retaining all rows — unlike GROUP BY which collapses them. This is an advanced SQL technique widely used in analytics.

Query 10 — Index Creation for Performance + Route ID 4 Query

Objective:

Create an index on route_id to speed up queries, then retrieve passengers on Route ID 4.

SQL Code:

```
-- Step 1: Create an index for performance
CREATE INDEX idx_route_id
ON passengers_on_flights(route_id);

-- Step 2: Query passengers on Route 4 (now uses index)
SELECT *
FROM passengers_on_flights
WHERE route_id = 4;
```

Result:

✓ Index created successfully. Passengers on Route ID 4 retrieved with improved query speed.

Observation:

💡 Indexes act like a book's index — MySQL uses them to jump directly to matching rows instead of scanning the entire table. Essential for large datasets to improve query performance.

Query 11 — Execution Plan for Route ID 4 (EXPLAIN)

Objective:

View the query execution plan to verify the index is being used for Route ID 4.

SQL Code:

```
EXPLAIN SELECT *
FROM passengers_on_flights
WHERE route_id = 4;
```

Result:

✓ EXPLAIN output shows 'type: ref' and 'key: idx_route_id', confirming the index is being used successfully.

Observation:

💡 EXPLAIN is a diagnostic tool that shows HOW MySQL executes a query. 'type: ref' means MySQL is using the index, which is far more efficient than a full table scan ('type: ALL').

Query 12 — Total Ticket Price per Customer per Aircraft (ROLLUP)**Objective:**

Calculate the total ticket price per customer per aircraft ID, including subtotals and grand total using ROLLUP.

SQL Code:

```
SELECT customer_id,
       aircraft_id,
       SUM(no_of_tickets * price_per_ticket) AS total_price
FROM ticket_details
GROUP BY customer_id, aircraft_id WITH ROLLUP;
```

Result:

✓ Results show per-customer per-aircraft totals, subtotals per customer (aircraft_id = NULL), and an overall grand total (both NULL).

Observation:

💡 WITH ROLLUP is an advanced aggregation extension that automatically adds subtotal and grand total rows. NULL values in output represent rollup summary rows, not missing data.

Query 13 — Business Class View**Objective:**

Create a database view showing only Business class customers and their airline brand.

SQL Code:

```
CREATE VIEW vw_business_class AS
SELECT customer_id,
       brand,
       class_id,
       price_per_ticket
FROM ticket_details
WHERE class_id = 'Business';

-- Query the view
SELECT * FROM vw_business_class;
```

Result:

✓ View 'vw_business_class' created. 13 rows returned showing Business class customers with their airline brand.

Observation:

💡 Views act as virtual tables built from a saved query. They simplify complex queries, improve security by limiting data exposure, and make reports reusable without rewriting SQL.

Query 14 — Stored Procedure — Routes Over 2000 Miles

Objective:

Create a stored procedure that extracts all route details where the traveled distance exceeds 2000 miles.

SQL Code:

```
DELIMITER $$
CREATE PROCEDURE get_long_routes()
BEGIN
    SELECT *
    FROM routes
    WHERE distance_miles > 2000;
END $$
DELIMITER ;

-- Execute the procedure
CALL get_long_routes();
```

Result:

✓ 24 routes with distance > 2000 miles returned. Procedure executes successfully.

Observation:

💡 Stored procedures encapsulate reusable SQL logic in the database. They reduce code repetition, improve security (can restrict direct table access), and enable parameterized queries.

Query 15 — Total Tickets and Spending per Customer (GROUP BY)

Objective:

Using GROUP BY, determine the total number of tickets purchased and total amount paid by each customer.

SQL Code:

```
SELECT customer_id,
       SUM(no_of_tickets) AS total_tickets,
       SUM(no_of_tickets * price_per_ticket) AS total_spent
FROM ticket_details
GROUP BY customer_id
ORDER BY total_spent DESC;
```

Result:

✓ All customers ranked by total spending. Highest spender and ticket count identified for loyalty program targeting.

Observation:

💡 GROUP BY collapses multiple rows per customer into one summary row. ORDER BY total_spent DESC helps identify VIP and high-value customers for personalized loyalty programs and offers.

Query 16 — Average Passengers per Flight Route

Objective:

Calculate the average number of passengers across all flight routes.

SQL Code:

```
SELECT AVG(passenger_count) AS avg_passengers_per_route
FROM (
    SELECT route_id,
```

```

        COUNT(*) AS passenger_count
    FROM passengers_on_flights
    GROUP BY route_id
) AS route_summary;

```

Result:

✓ avg_passengers_per_route = 1.5625 | 1 row returned in 0.000 sec. On average, 1.56 passengers travel per flight route across all active routes.

Observation:

💡 A subquery (derived table) first groups passengers by route and counts them. The outer query then averages those counts. Average of 1.5625 indicates most routes are underutilized — Air Cargo can optimize by using smaller aircraft on low-traffic routes.

Query Summary & SQL Techniques Used

#	Query	SQL Technique	Result	Status
1	Passengers on Routes 1-25	WHERE + BETWEEN	~25 rows	Correct
2	Business class revenue	SUM(), WHERE filter	Rs.6,034 13 tickets	Correct
3	Customer full name	CONCAT()	50 rows	Correct
4	Registered + ticketed customers	INNER JOIN + DISTINCT	41 unique customers	Correct
5	Emirates customers	JOIN + WHERE + DISTINCT	14 unique customers	Correct
6	Economy Plus (subquery)	Subquery + IN()	All Economy Plus listed	Correct
7	Revenue check	IF() + SUM()	Rs.15,369 Target Achieved	Correct
8	Create & grant user	CREATE USER + GRANT	User created	Correct
9	Max price per class	Window Function MAX() OVER()	Max per class shown	Correct
10	Index + Route 4 query	CREATE INDEX	Faster retrieval	Correct
11	Execution plan	EXPLAIN	Index confirmed used	Correct
12	Total price per customer (ROLLUP)	GROUP BY WITH ROLLUP	Subtotals + Grand total	Correct
13	Business class view	CREATE VIEW	13 rows in view	Correct
14	Routes > 2000 miles (stored proc)	CREATE PROCEDURE	24 long-haul routes	Correct
15	Tickets & spending per customer	GROUP BY + ORDER BY	Customers ranked by spend	Correct
16	Average passengers per route	Subquery + AVG()	1.5625	Correct

Key Business Insights

Insight	Finding	Business Impact
Total Revenue	Rs.15,369 — exceeds Rs.10,000 operational target	Company is profitable above target by 53%
Business Class Revenue	Rs.6,034 (~39% of total) from just 13 tickets	Premium pricing drives significant revenue
Long-Haul Routes	24 routes exceed 2,000 miles	Require larger aircraft allocation
Active Customers	41 out of 50 registered customers purchased tickets	9 inactive customers for win-back campaigns
Emirates Customers	14 unique customers traveled with Emirates	Key airline partner for targeted campaigns
Avg Passengers/Route	1.5625 passengers per route on average	Routes underutilized — optimize aircraft size

Conclusion

This project successfully demonstrates the application of both foundational and advanced SQL techniques on a real-world aviation dataset. Using MySQL Workbench, all 16 queries were designed, executed, and validated against the Air Cargo dataset.

SQL Concepts Applied:

Data Retrieval — SELECT, WHERE, BETWEEN, DISTINCT, CONCAT

Joins and Subqueries — INNER JOIN, IN() with subquery

Aggregation — SUM(), COUNT(), AVG(), GROUP BY, ORDER BY

Advanced Features — Window Functions, ROLLUP, Views, Stored Procedures, Indexes

Administration — User creation, GRANT, EXPLAIN execution plans

The insights derived from this analysis provide Air Cargo with actionable intelligence to improve route planning, customer targeting, and revenue optimization — making it a stronger competitor in the aviation industry.

Prepared by: Anushua Ghosh | Tool: MySQL Workbench | April 2026